

FICHE DE TD 3/3 • DIFFICILE

Informatique • Tle TI

Modélisation structurelle des systèmes d'information avec UML (diagrammes de classes et d'objets)

Module 3 : Systèmes d'information et bases de données

Cette fiche propose des problèmes de synthèse mobilisant l'ensemble des concepts de modélisation structurelle UML (classes, objets, relations, multiplicités, héritage, agrégation/composition) dans des contextes réalistes et contraignants.

EXERCICE 1 Modélisation d'un système de gestion de chantiers BTP

Une entreprise de BTP souhaite informatiser le suivi de ses chantiers. Un Chantier est identifié par un code, une adresse, une date de début prévisionnelle et une date de fin réelle. Il est géré par un unique ConducteurTravaux (employé de l'entreprise) et fait intervenir plusieurs SousTraitants externes. Un Ouvrier (employé) est affecté à un seul chantier à la fois. Chaque intervention d'un sous-traitant ou d'un ouvrier sur un chantier génère un RapportJournalier (date, description, heures effectuées). Un ConducteurTravaux supervise plusieurs chantiers. Un SousTraitant possède un SIRET, une raison sociale et une spécialité. L'entreprise gère un stock de Matériel (engins, outillage) qui peut être affecté à un chantier (composition forte : le matériel appartient à l'entreprise et sa durée de vie est liée à elle) ou loué ponctuellement auprès d'un fournisseur (association faible).

1. Identifier les classes pertinentes (au maximum 5 classes principales hors énumérations) et pour chacune : attributs (avec visibilité), opérations significatives (signature complète : visibilité, paramètres, type de retour). Justifier la visibilité choisie pour les identifiants et les dates.
2. Préciser et justifier la nature de chaque relation (Association simple, Héritage, Agrégation, Composition) entre : Chantier/ConducteurTravaux, Chantier/Ouvrier, Chantier/SousTraitant, Chantier/Matériel (appartenant à l'entreprise), Chantier/Rapport-Journalier, Entreprise/Matériel. Indiquer les multiplicités exactes des deux côtés.
3. Proposer une hiérarchie d'héritage factorisant les points communs entre ConducteurTravaux, Ouvrier et SousTraitant (ex : `Intervenant`). Redéfinir le diagramme de classes en intégrant cette abstraction. Quel impact sur la relation Chantier/Intervenant ?
4. (2 pts) Écrire la contrainte OCL (Object Constraint Language) ou en langage naturel formel stipulant : « La date de fin réelle d'un chantier ne peut être antérieure à sa date de début prévisionnelle » et « La somme des heures des rapports journaliers d'un ouvrier sur un mois ne peut dépasser 160h ».

EXERCICE 2 Analyse critique et correction d'un diagramme de classes

On propose le diagramme de classes suivant pour une application de E-commerce (description textuelle) :

- Client (1) ->(0..*) Commande : « passe »
 - Commande (1) ->(1..*) LigneCommande : « contient »
 - LigneCommande (1) ->(1) Produit : « référence »
 - Produit (1) ->(1) Categorie : « appartient »
 - Client (1) ->(0..1) Panier : « possède » (Agrégation)
 - Panier (1) ->(0..*) LignePanier : « compose » (Composition)
 - LignePanier (1) ->(1) Produit : « référence »
 - Commande hérite de Paiement (attributs : montant, date, mode).
 - Produit a un attribut -stock: int et une méthode +baisserStock(qte: int): bool.
1. Identifier quatre erreurs conceptuelles ou notationnelles majeures dans ce modèle (héritage abusif, multiplicités incohérentes, confusion agrégation/composition, visibilité/accès). Justifier chaque correction.
 2. Redessiner (décrire textuellement) la structure corrigée pour la gestion du paiement (Pattern Strategy ou classe associée) et pour la relation Panier/Client (cycle de vie).
 3. La méthode `baisserStock` est appelée depuis `LigneCommande` lors de la validation. Proposer une signature robuste (exceptions, atomicité) et expliquer pourquoi la visibilité de `stock` doit rester `private`. Comment modéliser une rupture de stock au niveau du diagramme de classes (contrainte d'invariant)?
 4. (2 pts) Différencier clairement, avec un exemple concret issu de ce domaine, la Composition et l'Agrégation en termes de cycle de vie, de responsabilité de suppression et de multiplicité du côté « tout ».

EXERCICE 3 Diagramme d'objets et simulation dynamique

On considère le modèle de classes suivant (simplifié) pour une Bibliothèque :

```
class Adherent { -id: String; -nom: String; -nbEmpruntsEnCours: int = 0; +emprunter(): bool; +
class Exempleire { -codeBarre: String; -etat: EtatExempleire {DISPONIBLE, EMPRUNTE, PERDU}; +c
class Emprunt { -dateDebut: Date; -dateFinPrevue: Date; -dateFinReelle: Date?; +estEnRetard():
class Bibliotheque { +catalogue: Set<Exempleire>; +adherents: Set<Adherent>; +empruntsEnCours:
Associations :
Adherent (1) -- (0..3) Emprunt (role: empruntsEnCours) -- (1) Exempleire (role: exempleire)
Bibliotheque (1) -- (0..*) Exempleire (Composition, role: catalogue)
Bibliotheque (1) -- (0..*) Adherent (Composition, role: adherents)
```

Bibliotheque (1) -- (0..*) Emprunt (Composition, role: empruntsEnCours)

Règles métier : Un adhérent ne peut avoir plus de 3 emprunts simultanés. Un exemplaire perdu ne peut plus être emprunté. Un emprunt en retard bloque les nouveaux emprunts de l'adhérent.

1. À l'instant t_0 , la bibliothèque contient : Adhérent A1 (Dupont, 0 emprunts), Exemplaires E1 (DISPONIBLE), E2 (EMPRUNTE), E3 (PERDU). Dessiner (ou décrire exhaustivement sous forme de tableau d'instances) le **diagramme d'objets** complet à t_0 , y compris les valeurs d'attributs et les liens.
2. Scénario : A1 emprunte E1 (création Emprunt Em1, dates : aujourd'hui, +14 jours). Puis A1 emprunte E2 (échec). Puis A1 rend E1 (clôture Em1). Dessiner l'état des objets (diagramme d'objets) à t_1 (après emprunt E1), t_2 (après tentative E2), t_3 (après retour E1). Préciser les valeurs de `nbEmpruntsEnCours`, `etat`, `dateFinReelle`.
3. (2 pts) Analyser la cohérence entre le diagramme de classes (multiplicité 0..3 côté Adherent/Emprunt) et la règle « blocage si emprunt en retard ». Cette multiplicité statique suffit-elle? Proposer une contrainte dynamique (pré/post-condition sur `emprunter()`) ou une modification structurelle (classe `StatutAdherent` ou état) pour garantir l'intégrité.

EXERCICE 4 Conception par contrat et patterns structurels

On souhaite modéliser le noyau d'un éditeur de documents structurés (type XML/JSON) où un `Noeud` peut être un `Element` (balise, attributs, enfants) ou un `Texte` (contenu brut). Un `Element` contient une liste ordonnée de `Noeud` (récursivité). On doit supporter des opérations transverses : `exporterXML()`, `validerSchema()`, `compterMots()` sans modifier les classes `Noeud`, `Element`, `Texte` à chaque nouvelle opération.

1. Proposer un diagramme de classes utilisant le pattern Composite pour la structure arborescente et le pattern Visiteur pour les opérations. Préciser les interfaces, l'héritage, les associations (composition/récursivité), les multiplicités et les signatures des méthodes `accepter(v: Visiteur)` et `visiterElement(e: Element)`, `visiterTexte(t: Texte)`.
2. Justifier pourquoi la relation `Element` – `Noeud` (enfants) est une Composition et non une Agrégation. Que se passe-t-il si on supprime un `Element` parent?
3. Le pattern Visiteur brise l'encapsulation (accès aux enfants via `getEnfants()`). Proposer une alternative (ex : Interpréteur ou méthode virtuelle `exporterXML()` dans chaque classe) et comparer les deux approches au regard du principe Ouvert/Fermé (OCP) et de la cohésion.
4. (2 pts) Dessiner un **diagramme d'objets** représentant l'instance mémoire du fragment XML : `<section><titre>Intro</titre><para>Texte<br>gras</br>.</para></section>`. Monter explicitement les liens de composition

réursive, les objets `Texte` distincts pour « Texte » et « gras » et « . », et l'ordre de la liste d'enfants.